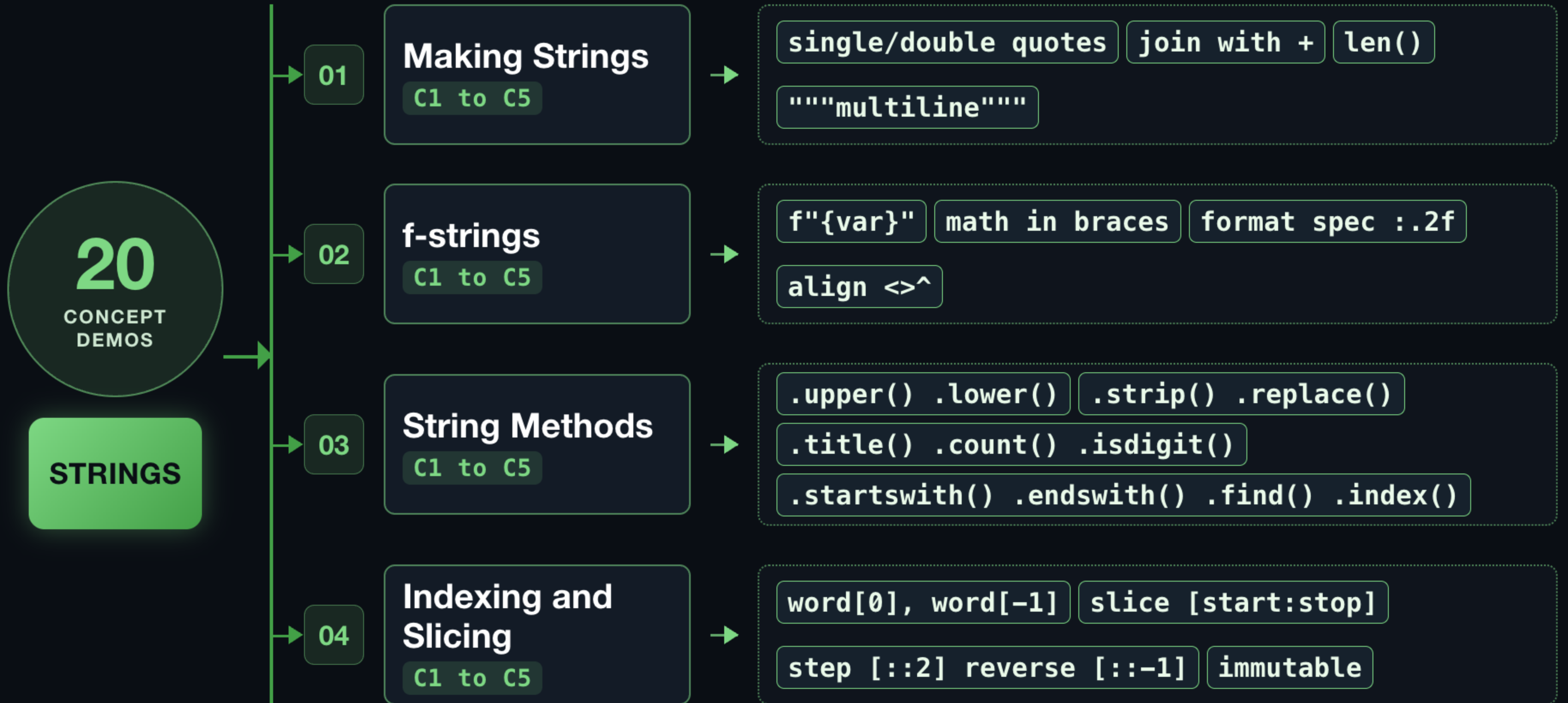


Section 03: Strings and f-strings **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Strings: Making Strings

A **string** is a piece of text wrapped in quotes; you can use single, double, or triple quotes, join strings with +, and count characters with len().

#	CONCEPT	SAMPLE CODE
1	Text in quotes " " ' ' Single or double quotes both make a string.	<code>first_name = "Kalyan"</code>
2	Quote inside the text Use the other quote style so inner quotes work.	<code>sentence = "It's a sunny day"</code>
3	Join strings + The + operator joins strings (concatenation).	<code>full_name = "Kalyan" + " " + "Reddy"</code>
4	Count characters len() len() gives how many characters are in a string.	<code>print(len("Python"))</code>
5	Multiline string """...""" Triple quotes keep text across several lines.	<code>bio = """Kalyan Reddy ... Hyderabad"""</code>

Strings: Building Text with f-strings

An **f-string** has the letter **f** before the quotes and drops variables, expressions, and formatting straight into the text with **{braces}**.

#	CONCEPT	SAMPLE CODE
1	Drop values into text <code>f"..."</code> Put f before the quotes, then use {variable}.	<code>print(f"My name is {name} and I am {age}")</code>
2	Math inside braces <code>{ }</code> Run an expression right inside the braces.	<code>print(f"Next year I will be {age + 1}")</code>
3	Fixed decimals <code>:.Nf</code> A colon plus .Nf sets the number of decimal places.	<code>f"{price:.2f}"</code>
4	Thousands separator <code>[:,.2f</code> A comma groups the digits into thousands.	<code>f"{price:[:,.2f}"</code>
5	Percentage format <code>:.N%</code> A percent sign scales the value and adds the % sign.	<code>f"{ratio:.1%}"</code>

Strings: f-string Number Formatting

After a colon inside the braces, a format spec controls how a number looks - decimals, commas, percent.

Decimal places

`:.Nf` rounds to N decimals: `.3f`, `.2f`, `.1f`, `.0f`.

Thousands

`:,` groups thousands, like `1,234.43`.

Percent

`:.N%` shows a fraction as a percent, like `87.5%`.

FORMAT SPECS

```
price = 1234.4321
```

```
f"{price:.2f}"      # 1234.43
```

```
f"{price:.0f}"      # 1234
```

```
f"{price:,.2f}"     # 1,234.43
```

```
ratio = 0.875432
```

```
f"{ratio:.1%}"      # 87.5%
```

A format spec changes how the number LOOKS (and gives back text) - the value itself is unchanged.

Strings: String Methods

A **method** is an action you call on a string with a dot, like `text.upper()`, and it returns a new string.

#	CONCEPT	SAMPLE CODE
1	Trim and change case <code>.strip()</code> <code>.upper()</code> <code>.replace()</code> Remove edge spaces, switch case, or swap text.	<pre>print(name.strip().upper())</pre>
2	Title case <code>.title()</code> Capitalize the first letter of every word.	<pre>print("kalyan reddy".title())</pre>
3	Count matches <code>.count()</code> Count how many times a piece of text appears.	<pre>print("kalyan reddy".count("a"))</pre>
4	Digits-only check <code>.isdigit()</code> True only when every character is a digit.	<pre>print("30".isdigit())</pre>
5	Check ends, locate text <code>.startswith()</code> <code>.endswith()</code> <code>.find()</code> Test the start or end, or find where text sits.	<pre>print(email.startswith("kalyan"))</pre>

Strings: Indexing and Slicing

Indexing reads one character by its position, and **slicing** reads a range of characters using square brackets.

#	CONCEPT	SAMPLE CODE
1	Index one character <code>[i]</code> Each character has a position, starting at 0.	<pre>print(word[0])</pre>
2	Slice a range <code>[start:stop]</code> start is included, stop is not.	<pre>print(word[0:3])</pre>
3	Slice with a step <code>::step</code> Step over characters; a step of -1 reverses.	<pre>print(word[::-1])</pre>
4	Negative index <code>[-N:]</code> Count from the end; take the last N characters.	<pre>print(plate[-4:])</pre>
5	Strings are immutable Read a character, but you cannot change one in place.	<pre>print("J" + word[1:])</pre>